

DAY 27 ASSIGNMENT INJECTION — SECURE INPUT HANDLING

Name:- Rishabh Raj

1. What is Injection?

Injection is a security weakness in which untrusted input is interpreted by an interpreter or component as part of a command or query. Instead of being treated only as data, the input can influence the instructions executed by the application.

2. What is untrusted input?

Untrusted input is any data supplied from a source that the application cannot automatically assume is safe. Examples include values from users, browsers, API clients, cookies, HTTP headers, uploaded files, and external systems.

3. Give five examples of user-controlled input.

Five examples are: (1) username or email fields, (2) search-box text, (3) URL or query parameters, (4) comments or form messages, and (5) uploaded file names or metadata. Other examples include HTTP headers, cookies, and API request fields.

4. What is an interpreter?

An interpreter or interpreting component processes data according to a language or command syntax and performs the resulting instructions. Examples include SQL database engines, operating-system command shells, template engines, and expression evaluators.

5. Explain the difference between Data and Instructions.

Data is information that an application is supposed to process, while instructions tell an interpreter what operation to perform. Injection occurs when untrusted data is allowed to change the intended instructions. Secure input handling keeps the two roles separate.

6. What is SQL Injection at a conceptual level?

SQL Injection occurs when untrusted input changes the meaning of a database query. If an application constructs SQL by directly combining user input with query text, an attacker may influence the query and potentially access, modify, or delete data. Parameterized queries help prevent this.

7. What is OS Command Injection at a conceptual level?

OS Command Injection occurs when an application passes untrusted input into an operating-system command in a way that allows the input to influence command execution. The safer approach is to avoid shell interpretation where possible, use dedicated APIs, and strictly validate allowed values.

8. Why is raw string concatenation with untrusted input risky?

Raw concatenation mixes untrusted data with program instructions. Special characters or syntax in the input may change how an interpreter parses the final command or query.

This can lead to injection, unauthorized actions, or data exposure.

9. What is a parameterized query?

A parameterized query separates SQL instructions from user-supplied values. The SQL structure is defined separately and input values are supplied as parameters, so the database treats those values as data rather than executable SQL syntax.

10. What is a prepared statement?

A prepared statement is a database statement whose structure is prepared separately from the values supplied to it. Parameters are bound to placeholders when the statement is executed. This separation helps prevent SQL injection.

11. Why do parameterized queries improve security?

They keep user-controlled values separate from SQL syntax. Therefore, input that contains SQL metacharacters is treated as a value rather than changing the intended query structure. This is one of the primary defenses against SQL injection.

12. What is input validation?

Input validation is the process of checking that incoming data meets the expected rules before the application uses it. Validation can check type, length, format, range, and allowed values.

13. What is allowlist validation?

Allowlist validation accepts only values that are explicitly permitted. For example, if a field should contain a student ID consisting of a fixed format, the server can accept only values matching that defined format and reject everything else.

14. Why is client-side validation alone not sufficient?

Client-side validation runs in an environment controlled by the user, so it can be bypassed or modified. An attacker can send requests directly to the server without using the normal browser interface. Therefore, client-side validation is useful for usability but cannot be the primary security control.

15. Why should important validation also happen server-side?

The server is the trusted enforcement point for application rules. Server-side validation ensures that requests are checked even when a client bypasses browser controls or sends specially crafted requests directly to the backend.

16. Explain Least Privilege for a database account.

The database account used by an application should have only the permissions required for its functions. For example, a student-search application that only needs to read specific records should not use an account with unrestricted database administration, table modification, or server-level privileges.

17. Why should detailed database errors not normally be displayed publicly?

Detailed database errors can reveal SQL statements, table names, column names, database technology, file paths, or other internal information. This information can help attackers understand the application and find weaknesses. Users should receive safe generic errors while detailed information is recorded securely in protected logs.